

Prueba de Caja Blanca

REQ009 ACTUALIZACION DE INFORMACION

“Sistema automatizado para la administración de ingresos en el conjunto Fortín del Valle”

Integrantes:
Joshep Chisaguano
Cristian Suquillo
Melanie Zuñiga
Paul Villacis

Fecha: 2026/07/06

Requisito funcional N° 6 Actualización de Información

1. CÓDIGO FUENTE

RESIDENTES

```
String error = validaciones.validarResidente(actualizado);  
if (error != null) { msg(error); return; }
```

```
if (!cedula.equals(cedulaOriginal) && repo.cedulaExiste(cedula, cedulaOriginal)) {  
    msg("La cédula " + cedula + " ya está registrada en otro residente."); return;  
}
```

```
String nuevaVivienda = (String) comboVivienda.getSelectedItem();
```

```
if ("Activo".equals(nuevoEstado)) {
```

```
    String ocupadaPor = repo.casaOcupadaPorActivo(nuevaVivienda, cedulaOriginal);
```

```
    if (ocupadaPor != null) {
```

```
        msg("No se puede activar este residente en la Casa N° " + nuevaVivienda
```

```
            + " porque ya está asignada a:\n" + ocupadaPor
```

```
            + "\n\nOpciones:\n • Cambie primero el estado del otro residente a 'Cancelado'\n • O
```

```
asigne una casa diferente a este residente.");
```

```
        return;
```

```
    }
```

```
}
```

```
if (!telMovil.isEmpty()) {
```

```
    String duploMovil = repo.telefonoMovilExiste(telMovil, cedulaOriginal);
```

```
    if (duploMovil != null) {
```

```
        msg("El teléfono móvil " + telMovil + " ya está registrado en:\n" + duploMovil); return;
```

```
    }
```

```
}
```

```
if (!telConv.isEmpty()) {
```

```
    String duploConv = repo.telefonoConvExiste(telConv, cedulaOriginal);
```

```
    if (duploConv != null) {
```

```
        msg("El teléfono convencional " + telConv + " ya está registrado en:\n" + duploConv); return;
```

```
    }
```

```
}
```

```
for (String[] v : vehiculos) {
```

```
    String placa = v[0].trim().toUpperCase();
```

```
    if (!placa.isEmpty()) {
```

```
        String duploPlaca = repo.placaExiste(placa, cedulaOriginal);
```

```
        if (duploPlaca != null) {
```

```
            msg("La placa " + placa + " ya está registrada en:\n" + duploPlaca); return;
```

```
        }
```

```
    }
```

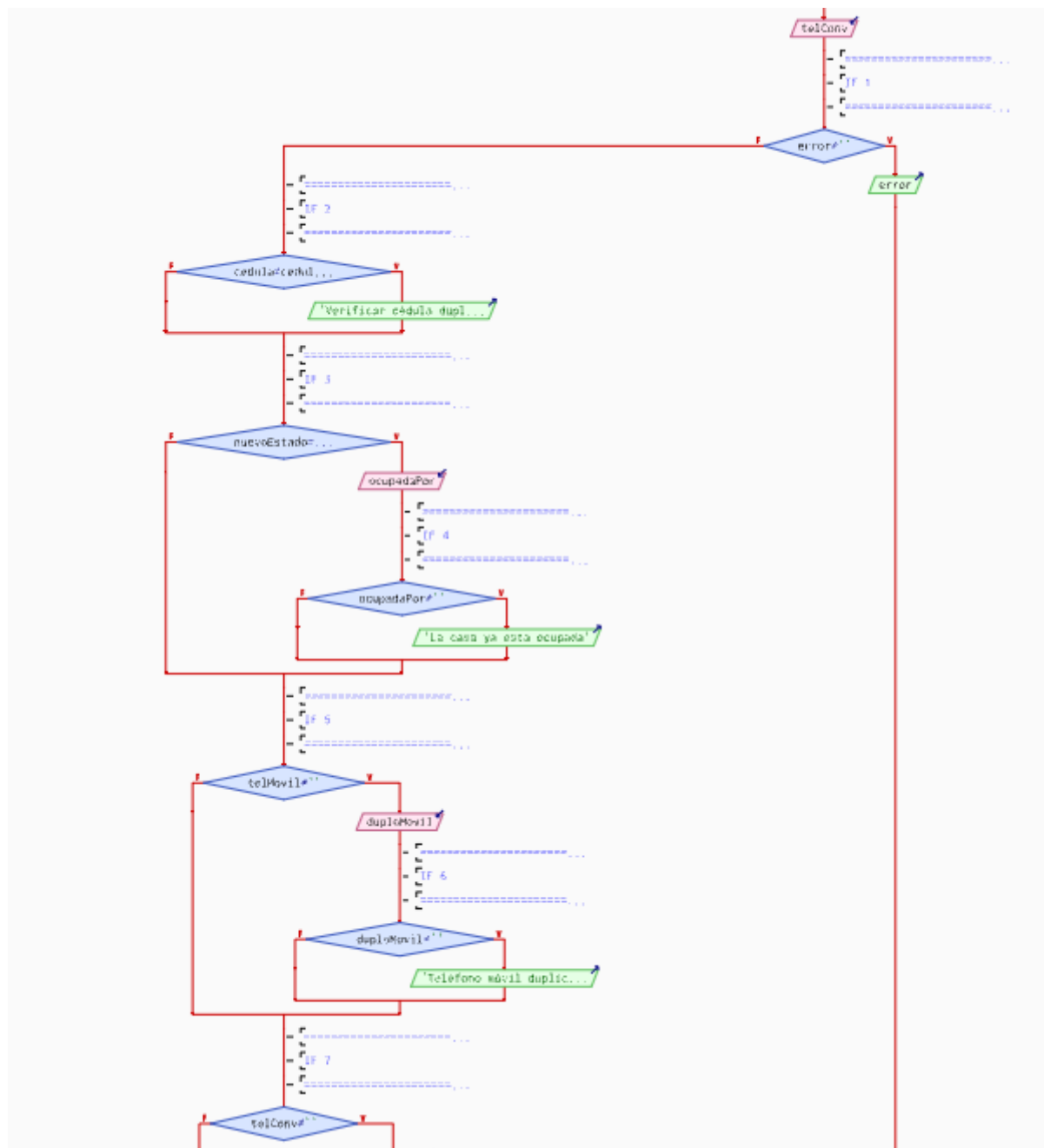
```
}
```

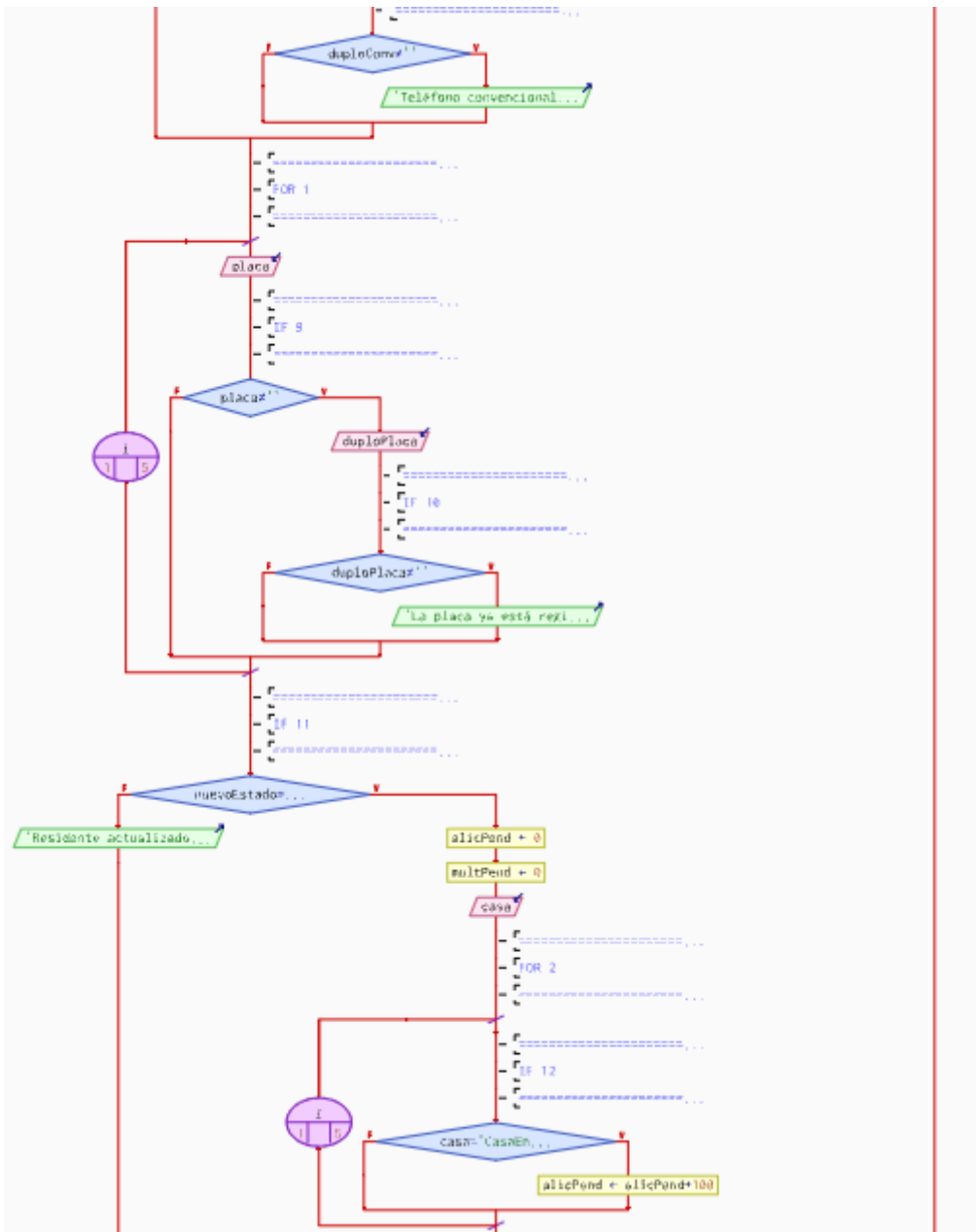
```

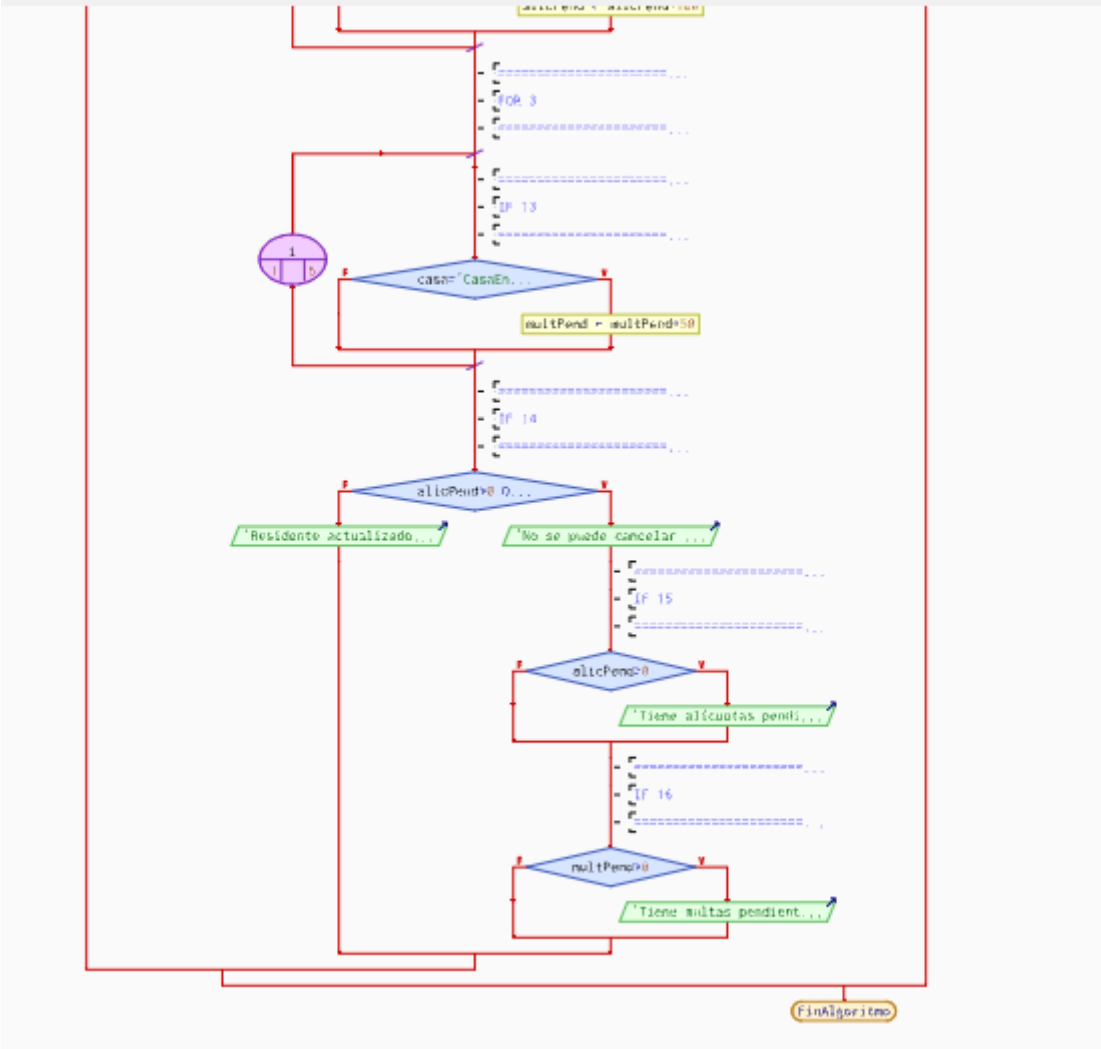
        if ("Cancelado".equals(nuevoEstado) &&
!"Cancelado".equals(residenteOriginal.getEstadoResidente())) {
            Modelo.AlmacenarAlicuotas repoAlic = new Modelo.AlmacenarAlicuotas();
            Modelo.AlmacenarMultas repoMult = new Modelo.AlmacenarMultas();
            String casa = (String) comboVivienda.getSelectedItemAt();
            StringBuilder deudas = new StringBuilder();
            double alicPend = 0, multPend = 0;
            for (Modelo.Alicuota a : repoAlic.obtenerTodas()) {
                if (casa.equals(a.getNumeroCasa()) &&
("Pendiente".equals(a.getEstado()) || "Atrasado".equals(a.getEstado()))
                alicPend += a.getMonto();
            }
            for (Modelo.Multa m : repoMult.obtenerTodas()) {
                if (casa.equals(m.getNumeroCasa()) && "Pendiente".equals(m.getEstado()))
                multPend += m.getMonto();
            }
            if (alicPend > 0 || multPend > 0) {
                String msj = "No se puede cancelar este residente porque tiene deudas pendientes:\n\n";
                if (alicPend > 0) msj += " • Alícuotas pendientes/atrasadas: $" + String.format("%.2f",
alicPend) + "\n";
                if (multPend > 0) msj += " • Multas pendientes: $" + String.format("%.2f", multPend) + "\n";
                msj += "\nRegularice los pagos antes de cancelar el residente.";
                msg(msj); return;
            }
        }
    }
}

```

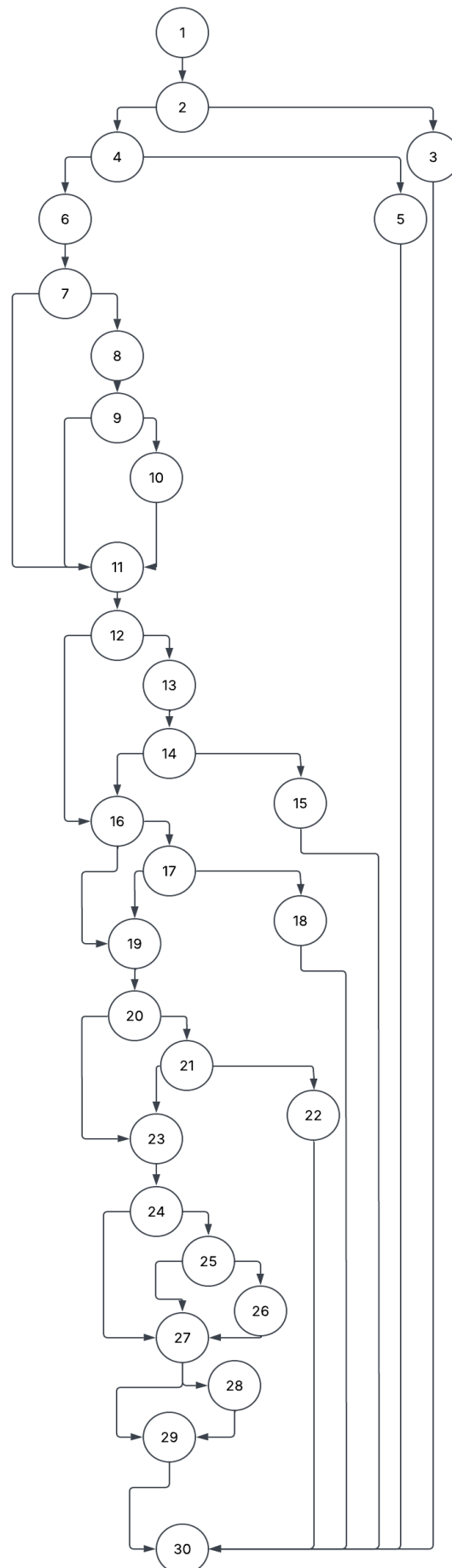
2. DIAGRAMA DE FLUJO (DF) RESIDENTES







3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

RUTAS

Ruta	Secuencia
R1	1->2->3->30
R2	1->2->4->5->30
R3	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->21->23->24->25->27->30
R4	1->2->4->6->7->10->11->12->13->14->16->17->19->20->21->23->24->30
R5	1->2->4->6->7->8->9->10->11->12->13->14->15->30
R6	1->2->4->6->7->8->9->10->11->12->13->14->16->17->18->30
R7	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->21->23->24->25->26->27->28->29->30
R8	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->21->22->30
R9	1->2->4->6->7->8->9->11->12->13->14->16->17->19->20->21->23->24->25->27->28->29->30
R10	1->2->4->6->7->11->12->13->14->16->17->19->20->21->23->24->25->27->28->29->30
R11	1->2->4->6->7->8->9->10->11->12->16->17->19->20->21->23->24->25->27->28->29->30
R12	1->2->4->6->7->8->9->10->11->12->13->14->16->19->20->21->23->24->25->27->28->29->30
R13	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->23->24->25->27->28->29->30
R14	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->21->23->24->25->27->28->29->30
R15	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->21->23->24->27->28->29->30
R16	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->21->23->24->25->27->29->30
R17	1->2->4->6->7->8->9->10->11->12->13->14->16->17->19->20->21->23->24->25->27->28->29->30

5. COMPLEJIDAD CICLOMÁTICA eliminación de contratos

Se puede calcular de las siguientes formas:

DATOS:

P: Número de nodos predicado: 16

A: Número de aristas: 45

N: Número de nodos: 30

V(G) Nodos predicados (IF-THEN-ELSE) +1

$$V(G) = P + 1$$

$$V(G) = 17$$

$$V(G) = A - N + 2$$

$$V(G) = 45 - 30 + 2$$

$$V(G) = 17$$